

Experiment 5: Simulate a Cloud Scenario using CloudSim

Aim:

To simulate a cloud computing environment using CloudSim and implement a custom scheduling algorithm.

Steps:

Step 1: Install Java JDK

1. Download Java JDK 8 or later from:  <https://www.oracle.com/java/technologies/javase/javase8-archive-downloads.html>
2. Install it and set environment variables:
 - `JAVA_HOME`: Path to JDK folder
 - Add `JAVA_HOME\bin` to system PATH

Step 2: Install Eclipse IDE

1. Download Eclipse IDE:  <https://www.eclipse.org/downloads/packages/>
2. Extract and run `eclipse.exe`

Step 3: Download and Import CloudSim

1. Download CloudSim (v3.0.3) from:  <http://www.cloudbus.org/cloudsim/>
2. Extract the ZIP file
3. Open Eclipse → File → New → Java Project → Name: `CloudSimProject`
4. Import `cloudsim-3.0.3/src` files into `src` folder of the project

Step 4: Add Required JAR Files

1. Right-click the project → Build Path → Configure Build Path
2. Under **Libraries** → Add External JARs → Add all `.jar` files from:
 - `cloudsim/lib/`

Sample CloudSim Simulation Code

Create a class `CloudSimTest.java` and use the following code:

```
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import java.util.*;

public class CloudSimTest {
    public static void main(String[] args) throws Exception {
        int numUsers = 1;
        Calendar calendar = Calendar.getInstance();
```

```

        CloudSim.init(numUsers, calendar, false);
        Datacenter datacenter = createDatacenter("Datacenter_1");
        DatacenterBroker broker = new DatacenterBroker("Broker");
        int brokerId = broker.getId();
        Vm vm = new Vm(0, brokerId, 1000, 1, 512, 1000, 1000, "Xen", new
CloudletSchedulerPriority());
        broker.submitVmList(Collections.singletonList(vm));
        Cloudlet cloudlet1 = createCloudlet(0, brokerId, 3); // Priority 3
        Cloudlet cloudlet2 = createCloudlet(1, brokerId, 1); // Priority 1
        Cloudlet cloudlet3 = createCloudlet(2, brokerId, 2); // Priority 2
        List<Cloudlet> cloudletList = new ArrayList<>();
        cloudletList.add(cloudlet1);
        cloudletList.add(cloudlet2);
        cloudletList.add(cloudlet3);
        broker.submitCloudletList(cloudletList);
        CloudSim.startSimulation();
        CloudSim.stopSimulation();
        List<Cloudlet> list = broker.getCloudletReceivedList();
        for (Cloudlet c : list) {
            System.out.println("Cloudlet ID: " + c.getCloudletId() + " Status: " + c.getStatus() + "
StartTime: " + c.getExecStartTime() + " FinishTime: " + c.getFinishTime());
        }
    }

    private static Datacenter createDatacenter(String name) throws Exception {
        List<Host> hostList = new ArrayList<>();
        List<Pe> peList = new ArrayList<>();
        peList.add(new Pe(0, new PeProvisionerSimple(1000)));

        hostList.add(new Host(0, new RamProvisionerSimple(2048), new
BwProvisionerSimple(10000),
            1000000, peList, new VmSchedulerTimeShared(peList)));

        return new Datacenter(name, new DatacenterCharacteristics("x86", "Linux", "Xen",
hostList,
            10.0, 3.0, 0.05, 0.1, 0.1, new LinkedList<>()),
            new VmAllocationPolicySimple(hostList), new LinkedList<>(), 10);
    }

    private static Cloudlet createCloudlet(int id, int userId, int priority) {
        UtilizationModel utilizationModel = new UtilizationModelFull();
        Cloudlet cloudlet = new Cloudlet(id, 40000, 1, 300, 300,
            utilizationModel, utilizationModel, utilizationModel);
        cloudlet.setUserId(userId);
        cloudlet.setPriority(priority); // Custom priority
        return cloudlet;
    }
}

```

Custom Scheduler Code: Priority based Scheduling

CloudletSchedulerPriority.java

```
import org.cloudbus.cloudsim.*;
import java.util.*;

public class CloudletSchedulerPriority extends CloudletSchedulerTimeShared {
    @Override
    public double updateVmProcessing(double currentTime, List<Double> mipsShare) {
        if (cloudletExecList.isEmpty()) return 0.0;

        // Sort cloudlets by priority (higher priority first)
        cloudletExecList.sort(Comparator.comparingInt(Cloudlet::getPriority));

        return super.updateVmProcessing(currentTime, mipsShare);
    }
}
```

How to Run:

1. Add both Java files (`CloudSimTest.java` and `CloudletSchedulerPriority.java`) in your Eclipse project under `src`.
2. Right-click `CloudSimTest.java` → Run As → Java Application
3. Output will show execution order based on priority (priority 1 executes first).

Result:

Successfully simulated a cloud scenario using CloudSim and executed cloudlets with a priority-based custom scheduling algorithm.